AgentISA: Formal Instruction Set Design for LLM Agents

Anonymous Author(s)

Abstract

Large Language Models run as autonomous agents, yet ad-hoc prompt engineering and per-provider function calling leave their behavior difficult to specify, analyze, and optimize systematically. We present AgentISA, an instruction set architecture making agent behavior a first-class compilable artifact: (1) a formal instruction set—compositional semantics, a type system, pre/post-conditions—checked offline by a shipped bounded verifier for pure instructions, bracketed by runtime pre-dispatch validation; (2) Agentic Calling, lifting provider-specific function calling into reusable, cacheable instructions; (3) instruction learning that mines recurring trace patterns, promoting them as composite-replay instructions; (4) a compiler and runtime applying classical optimization—fusion, dead-code elimination, pipeline scheduling, dynamic hot-path specialization—to agent instruction streams.

A reproducible microbenchmark drives the real implementation against a mock LLM backend: result caching reduces simulated API calls by 59.5–79.6% on synthetic Zipf workloads (2,000 tasks, 5 seeds); static compilation cuts the effective instruction count by 34.2% (35.4% at length 1,000), decomposed by a same-seed ablation into fusion (21.6%) and dead-code elimination (24.5%) with an 11.9-point overlap; dynamic hot-path optimization shrinks repeatedly executed traces by 65.8%. Two real-gateway experiments extend validation: the cache reproduces computed hit/miss statistics at every skew level with measured wall-clock and token savings, and the compiled, hot-path-adopted stream issues zero new gateway calls with byte-identical artifacts (cache-mediated). A four-task system-level characterization (12.5–25.0% per-task compile shrink) calibrates the synthetic headline; 43 multi-agent tests provide functional validation. We position these results as controlled-setting evidence, not as production end-to-end measurements. The shipped prototype implements result caching, the benchmark-firing static passes, and threshold-triggered trace re-optimization; tree-pattern instruction selection, cycle-accurate pipeline scheduling, symbolic-equivalence checking, and capability metric extraction remain specified or stubbed—no claim rests on them.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

CCS Concepts

• **Software and its engineering** → >**Compilers**; • **Theory of computation** → >**Program reasoning**; • **Computing methodologies** → >**Artificial intelligence**; • **Computer systems organization** → >**Architectures**.

Keywords

Instruction Set Architecture; Large Language Models; Compiler Optimization; Multi-agent Systems; Formal Methods; Result Caching

ACM Reference Format:

Anonymous Author(s). 2026. AgentISA: Formal Instruction Set Design for LLM Agents. In . ACM, New York, NY, USA, 20 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Recent advances in Large Language Models (LLMs) have produced systems with strong understanding, generation, and—through instruction tuning and human feedback—instruction-following capabilities [1, 2, 3, 4, 5, 6], and a rapidly growing body of work couples them with tools, memory, and control loops to build autonomous agents [9, 10, 11, 12]. Yet the engineering substrate beneath these agents remains ad hoc: behaviors are specified as prompts and provider-specific function-calling schemas—difficult to specify precisely, analyze formally, or optimize systematically.

1.1 Background and Motivation

Function calling—selecting an API from a declared tool set and emitting structured arguments—has become the dominant interface between LLMs and external systems [13, 14, 15], consolidated by standardization efforts such as the Model Context Protocol [18] and measured by harnesses such as the Berkeley Function Calling Leaderboard [17]. However, the unit of interaction remains the individual API call: each call is dispatched, priced, and forgotten independently, even when an agent repeatedly executes structurally identical work.

This per-call statelessness contrasts with how computing systems historically handle repetitive execution: classical compilers and hardware translate recurring patterns into cheaper forms—instruction fusion, dead-code elimination, caching, trace-based dynamic optimization [33, 34, 45]. Our motivating hypothesis is that these techniques apply to LLM agent workloads: if agent behavior is expressed as an explicit instruction stream rather than opaque prompts, recurring patterns become visible to a compiler and optimizable with well-understood, semantics-preserving transformations.

1.2 Challenges in Current LLM-based Agent Systems

Five structural challenges motivate AgentISA:

1. **Per-call statelessness of tool interfaces.** Mainstream function-calling protocols treat each call as an isolated event [13, 14, 18]: the provider has no notion of a *program* being executed, repeated identical work incurs repeated cost, and caching, where present, is a bolt-on afterthought rather than a property of the execution model.

2. **Experience that does not accumulate as compiler-visible instruction structure.** Paradigms such as Reflexion [10] and Voyager [12] accumulate experience as verbal reflections or inference-time skill libraries—not reified as reusable, statically optimizable instructions that a compiler can fuse, schedule, or eliminate.

3. **Absence of a shared behavioral contract.** Each multi-agent framework defines its own abstractions for roles, messages, and workflows [19, 20, 21, 22, 23, 24]; no instruction-level contract spans frameworks, fragmenting tooling and cross-framework analysis.

4. **Cost pressure at the wrong layer.** FrugalGPT [28] shows LLM inference cost is a first-order concern, and serving systems attack it below the API boundary with paged attention, continuous batching, KV-cache-centric architectures, and attention kernels [38, 39, 40, 41]; what is missing is the complementary, above-the-API layer that reduces the *number and size* of calls an agent issues in the first place.

5. **Integration complexity across providers.** Portability is handled at the library level [26, 27]: logic written against one provider's function-calling surface does not transfer cleanly to another, and capability differences are hard to reason about systematically [17].

1.3 Research Objectives and Contributions

This paper introduces AgentISA, an instruction set architecture that addresses these challenges by making agent behavior a first-class compilable artifact. Our contributions are:

1. **A formal instruction set** for LLM agents with compositional semantics, a type system, and pre/post-conditions on every instruction (Section 3.1). Two deliberately modest sanity results are stated as design rationale: Turing completeness (Theorem 3.1)—negative certification that the instruction set does not constrain expressiveness—and type safety for the pure core (Theorem 3.2), which fixes the well-formedness obligation the offline bounded checker discharges for pure instructions (Section 3.3.3) and the runtime brackets with pre-dispatch context validation.

2. **Agentic Calling**, a mechanism lifting provider-specific function calling into reusable, cacheable, semantically specified agent instructions, with a model-adapter layer for cross-provider portability (Section 3.2).

3. **Trace-based instruction learning**, which mines recurring instruction patterns from interaction traces by frequency-based sequence mining and deterministic behavior grouping, and promotes validated recurring patterns into candidate instructions whose `execute` bodies replay the mined constituent subsequence by sequential composition over a shared context (Section 3.3; replay reproduces the recorded subsequence's outputs on the same inputs and does not generalize beyond them, as the section states).

4. **A compiler and runtime** implementing fusion, dead-code elimination, parallel-grouped scheduling, and dynamic hot-path specialization for agent instruction streams (Sections 3.4 and 4), together with specified tree-pattern instruction selection and cycle-accurate pipeline scheduling (Sections 4.1.1 and 4.1.2), with semantic-preservation arguments for the implemented rewrites (Theorems 3.4–3.6, proofs in Appendix C).

5. **An empirical evaluation** of the implemented system, with full methodology disclosure. On synthetic Zipf-distributed workloads (5 random seeds each), caching reduces simulated API calls by 59.5–79.6% depending on skew, static compilation reduces the effective instruction count by 34.2% ± 1.8% (sequence length 200), and dynamic optimization shrinks repeatedly executed traces by 65.8% ± 4.6%. Two real-gateway experiments (Sections 5.7 and 5.7.1) validate the cache and the compiled pipeline against a real LLM gateway, and a four-task system-level characterization (Section 5.8) composes the mechanisms on realistic (non-synthetic) instruction streams.

6. **Multi-agent coordination infrastructure** with capability-based agent selection, execution planning, fallback strategies, and a security monitor enforcing authentication, authorization, resource-access, and data-sensitivity policies (Sections 3.5 and 5.6), validated by 43 passing functional tests.

Our empirical claims are scoped accordingly: Sections 5.2–5.5 exercise the real `agentflow.core.isa` implementation against a mock LLM backend on synthetic workloads; Sections 5.7–5.8 extend the validation at reduced scale on realistic streams; none are production end-to-end measurements (threats to validity in Section 5.9).

Section 2 reviews related work; Sections 3 and 4 present the design and its implementation; Section 5 the empirical evaluation; Sections 6 and 7 discussion and conclusions.

2 Related Work

AgentISA draws on eight lines of work, reviewed in turn.

2.1 Foundation LLMs and Function Calling

Modern LLMs are autoregressive transformers trained at large scale [1, 2, 3, 4], with instruction tuning and reinforcement learning from human feedback making them reliable instruction followers [5, 6]. Their text distribution is conventionally factorized as:

$$P(y|x) = \prod_{t=1}^{T} P(y_t|x, y_{<t}; \theta)$$

where x is the input context, y the generated output, and θ the model parameters. Function calling constrains this distribution to a structured output space:

$$F(x) = \text{argmax}_{f \in \mathcal{F}} P(f|x, \mathcal{F})$$

where $\mathcal{F}$ is the set of declared functions and $F(x)$ the selected call. GPT-4 [3], PaLM [2], and open models such as LLaMA [4] expose this capability with provider-specific schemas; AgentISA does not modify this layer but treats any such interface as a *backend* targeted by a common instruction set, as a compiler targets multiple processor backends [33].

The cost structure of this layer motivates our work. If each call carries latency τ_i plus a context-switching overhead ω_i (prompt reconstruction, serialization, cache misses; the symbol δ is reused in Appendix B for per-instruction transition functions—the two are unrelated), then n similar calls cost:

$$C(n) = \sum_{i=1}^{n} (\tau_i + \omega_i)$$

Definition 2.1 (Function Call Overhead): The overhead of executing n function calls is $C(n)$, the sum of per-call latencies and per-call context reconstruction costs. When repeated calls carry identical semantic inputs and satisfy the cache-validity conditions of Section 3.2, the repeated context-construction cost ω_i is avoidable—which is precisely the cost AgentISA's caching and fusion are designed to remove. Structural similarity alone does not make a re-invocation waste: if an input's dependencies have changed or the call probes external state, re-execution has semantic meaning, and the cache-validity conditions—not structural identity—are what license reuse.

2.2 Reasoning and Agent Paradigms

A second line enhances reasoning at prompt time: chain-of-thought [7] and zero-shot reasoning [8]; ReAct's interleaved reasoning and action [9]; Reflexion's verbal self-critique [10]; Tree of Thoughts' deliberative search [11]; Voyager's open-ended skill acquisition [12]. These paradigms define *what* an agent should do at the cognitive level; AgentISA defines *how* the resulting behaviors are represented, compiled, and executed as instructions—a ReAct loop becomes an instruction sequence whose tool invocations are fused, cacheable agentic calls.

2.3 Tool Calling and Interface Standardization

Toolformer trains models to interleave tool calls with text [13]; Gorilla connects models to large API corpora [14]; ToolLLM scales tool coverage to 16,000+ real-world APIs [15]; HuggingGPT orchestrates model hubs [16]. On the standardization side, the Model Context Protocol specifies how tools and context are exposed [18], and the Berkeley Function Calling Leaderboard evaluates function-calling accuracy cross-model [17]. These efforts standardize the *interface*; AgentISA standardizes the *execution model above it*: an AgentISA program compiled for two providers differs only in the final translation step (Section 3.2.3), and interface-level standards alone cannot fuse or eliminate calls.

2.4 Multi-Agent Frameworks

AutoGen frames agent applications as multi-agent conversations [19]; MetaGPT assigns standardized roles and workflows [20]; AgentVerse facilitates multi-agent collaboration and studies emergent behaviors [21]; ChatDev organizes agents as a software-development organization [22]; CAMEL studies communicative agent societies [23]; AutoGPT popularized autonomous goal-driven agents [24]. AgentISA contributes a different artifact to this space: not a framework or orchestration philosophy but an instruction-level substrate—capability profiles, coordination instructions with barrier semantics, and a security monitor providing capability-based agent selection, per-stage fallback planning, and policy enforcement for authentication, authorization, resource access, and data sensitivity—that any such framework could target (Section 3.5, Section 5.6).

2.5 Prompt Compilation and Cost Optimization

Closest to our compilation perspective is DSPy, which compiles declarative prompt programs into optimized pipelines via prompt search and fine-tuning [25]; LangChain [26] and LlamaIndex [27] provide library-level chaining and indexing abstractions; FrugalGPT reduces cost through model cascades [28]. AgentISA differs in the level at which optimization operates. DSPy optimizes *prompts*; FrugalGPT optimizes *model selection*; AgentISA optimizes *instruction streams*: fusion, dead-code elimination, parallel grouping (the shipped form of pipeline scheduling), result caching, and dynamic hot-path specialization—implemented and measured (Sections 3.4, 4, and 5)—plus tree-pattern instruction selection over pattern covers and cycle-accurate pipeline scheduling, specified but not yet implemented (Sections 4.1.1 and 4.1.2). These classical compiler transformations [33] compose with—rather than replace—prompt-level and model-level optimization.

2.6 Benchmarks for Agent Systems

AgentBench evaluates LLMs as agents across diverse environments [29]; GAIA tests general assistant abilities [30]; PlanBench probes planning and reasoning about change [31]; SWE-bench measures resolution of real-world GitHub issues [32]. These benchmarks evaluate *end-to-end capability*; AgentISA's evaluation measures *subsystem behavior under controlled conditions*—call reduction, instruction-count reduction, trace shrinkage (Section 5). The two are complementary: instruction-level metrics can attribute end-to-end gains to specific mechanisms when such gains are measured, and are reproducible without live model access.

2.7 Classical Compiler and Architecture Techniques

AgentISA's optimization machinery is deliberately classical: tree-pattern instruction selection with dynamic programming, peephole optimization, and data-flow-based dead-code elimination [33]; pipelining, hazard detection, and resource-constrained scheduling [34]; the open, well-specified instruction-set discipline of RISC-V [35]; type safety via progress-and-preservation [36]; formally verified compiler correctness in the CompCert tradition [37]. Our contribution is not a new technique but the demonstration that this machinery transfers to LLM agent workloads, with the semantic-preservation arguments adapted to instructions with pre/post-conditions (Appendix C).

2.8 LLM Serving Systems

Below the API boundary, the serving community has made inference dramatically cheaper: PagedAttention [38]; Orca's continuous batching [39]; FlashAttention's IO-aware attention [40]; Mooncake's KVCache-centric disaggregation [41]; inference-cost energy analyses [42]. This line and AgentISA are orthogonal layers of the same stack: serving systems make each call cheaper; AgentISA reduces how many calls an agent makes and how much redundant work each carries—the two compound multiplicatively.

2.9 Research Gaps and Our Contributions

In summary, the literature leaves four gaps that AgentISA addresses:

1. **No instruction-level contract for agent behavior.** Interfaces are standardized [13, 14, 18] and frameworks numerous [19, 20, 21, 22, 23, 24], but no shared instruction set spans providers and frameworks.

2. **No compiler for agent behavior.** Prompt-level compilation exists [25] and model-level cost reduction exists [28], and *caching itself is well-trodden ground*—at the response level (GPTCache [46]), at the KV-prefix level (RadixAttention/SGLang [47]), and at the provider prompt-caching level. What none of these layers expose is an *instruction-level* contract: caching there is an optimization of model serving—invisible to the agent program's structure, and without effect on it. Instruction-level fusion, elimination, and scheduling—with cacheability as a declared property of the instruction set—have not, to our knowledge, been demonstrated.

3. **No semantics-preserving account of agent optimization.** Serving work verifies kernels and schedulers, not agent-level transformations; we provide progress-and-preservation-style arguments for ours (Theorems 3.2, 3.4, 3.5, 3.6).

4. **No reproducible instruction-level evaluation.** Existing benchmarks measure end-to-end capability [29, 30, 31, 32]; we contribute a seed-controlled evaluation: a microbenchmark suite with fully disclosed synthetic workloads and mock-backend instrumentation, extended by real-gateway and system-level validations (Section 5).

AgentISA addresses these four gaps directly—the instruction set and Agentic Calling abstraction (Sections 3.1–3.2), instruction-level optimization with semantic-preservation arguments (Section 3.4, Theorems 3.2 and 3.4–3.6, Appendices B–C), trace-derived instruction learning (Section 3.3), and a seed-controlled evaluation (Sections 5.2–5.8); the full contribution list is in Section 1.

3 AgentISA Design

Building on Sections 1 and 2, we present the AgentISA design: the core architecture (Section 3.1), the Agentic Calling framework (Section 3.2), trace-based instruction learning (Section 3.3), instruction organization and optimization (Section 3.4), and the capability-assessment layer (Section 3.5).

3.1 Core Architecture

3.1.1 Instruction Set Specification. The AgentISA instruction set is formally defined as a tuple:

Definition 3.1 (AgentISA Core):

$$\text{AgentISA} = (I, T, \Sigma, \Delta, \Phi)$$

where:

- I is the set of instructions
- T is the type system
- Σ is the state space
- Δ is the transition function set
- Φ is the verification function set

Following the conventions of classical instruction set design [33, 34, 35], we define the basic instruction format:

```
INSTRUCTION = {
    opcode: OPCODE,
    args: ARGS[],
    precondition: PRED,
    postcondition: PRED,
    context: CONTEXT
}
```

The instruction set is categorized into four fundamental classes:

1. **Control Flow Instructions**:
$$\mathcal{C} = \{seq, branch, loop, call\}$$

2. **State Management Instructions**:
$$\mathcal{S} = \{load, store, update, cache\}$$

3. **LLM Interaction Instructions**:
$$\mathcal{L} = \{query, response, context\}$$

4. **Resource Management Instructions**:
$$\mathcal{R} = \{allocate, free, monitor\}$$

Two sanity results anchor the definition; neither introduces new machinery [33, 36]: Theorem 3.1 certifies the instruction set does not constrain what agent programs can express, and Theorem 3.2 fixes the well-formedness obligation the offline bounded checker discharges for pure instructions (Section 3.3.3) and the runtime brackets with pre-dispatch context validation (Section 3.2.3).

Theorem 3.1 (Turing Completeness): The AgentISA instruction set is Turing complete.

Proof sketch: a translation from WHILE programs—known Turing equivalent—to instruction sequences, by simulation: state management instructions encode an unbounded store, control-flow instructions encode composition, branching, and iteration. The full construction is in Appendix A.

3.1.2 Type System and Semantics. The type system T extends the simply-typed discipline of Pierce [36] with dependent-type constructions (Π, Σ, μ):

Definition 3.2 (AgentISA Type System):

$$T ::= \text{Base} \mid \Pi(x : T_1).T_2 \mid \Sigma(x : T_1).T_2 \mid \mu X.T$$

where:

- Base types include {Bool, Int, String, Float}
- Π represents dependent function types
- Σ represents dependent pair types
- μ represents recursive types

The semantic rules follow operational semantics:

$$\frac{e_1 \to e_1'}{E[e_1] \to E[e_1']}\ \text{(Context)} \qquad \frac{}{(\lambda x.e)v \to e[x := v]}\ (\beta\text{-reduction})$$

The pure core carries a standard type-safety argument—progress and preservation, in the style of Pierce [36]—discharging the well-formedness role fixed in Section 3.1 and extended to the shipped instruction set by the bounded checker of Section 3.3.3:

Theorem 3.2 (Type Safety for the Pure Core): Well-typed programs in the pure core calculus of Appendix B—the λ-fragment (base types, dependent Π/Σ types, recursion) plus the composition operators applied to instructions carrying typed pre/post-conditions—cannot "go wrong", under the per-instruction validity obligation of Definition 3.5 (stated in Section 3.3.3 below; pre/post-conditions), with transition functions assumed total on precondition-satisfying states (Appendix B.7). Effectful instruction classes are embedded via their typed pre/post-conditions and covered to the extent their preconditions are checkable.

Proof sketch: By the standard two-lemma technique—progress and preservation—as presented in Pierce [36]:

1. Progress: $\forall e, T.\ \vdash e : T \implies \text{value}(e) \vee \exists e'.\ e \to e'$
2. Preservation: $\forall e, e', T.\ \vdash e : T \wedge e \to e' \implies \vdash e' : T$

Both are proved by induction over Appendix B's merged judgment system, with a joint argument for the composition cases: progress from the Definition 3.5 obligation plus totality, preservation from postcondition typing, composition via the chained invariant $post(i_1) \Rightarrow pre(i_2)$ (Appendix B).

3.1.3 LLM Integration Interface. The LLM integration interface is a protocol playing the same role for agent backends that the Model Context Protocol plays for tool exposure [18]: $\mathcal{P}_{LLM} = (Q, R, C, \psi)$, where Q is the query space (JSON Schema), R the response space (JSON Schema), C the context-management surface (today the context-window size), and ψ the state mapping Map<AgentState, LLMContext>—corresponding to the interface conventions of modern function-calling providers [3, 18].

3.2 Agentic Calling Framework

3.2.1 Function Calling to Agentic Calling Transformation. The transformation from function calling to Agentic Calling is formalized as:

Definition 3.3 (Agentic Transformation):

$$\mathcal{T} : F \to A$$

where F is the function call space and A is the agent instruction space.

$\mathcal{T}$ is required to preserve call semantics; this holds by construction for the three-step process below—the synthesized instruction records the function signature verbatim, embeds the context verbatim (Theorem 3.3), and delegates execution to the original call—so we state preservation as a property of the construction rather than a separately proved theorem:

$$\forall f \in F : \text{sem}(\mathcal{T}(f)) \equiv \text{sem}(f)$$

The transformation process follows three steps, mirroring the signature-to-call conventions of function-calling interfaces [13, 14]:

1. Function signature analysis
2. Context embedding
3. Instruction synthesis

3.2.2 Context Preservation and State Management. State management in AgentISA follows a formal state machine model:

$$M = (S, s_0, \delta, F)$$

where:

- S: Set of states
- s_0: Initial state
- δ: State transition function
- F: Set of final states

Extraction-time context preservation is guaranteed by construction (Theorem 3.3); at runtime it is only monitored:

Theorem 3.3 (Context Preservation): Let ϕ be the context interpretation function (defined compositionally over context *contents*—the keys and values the context carries), and t the extraction transformation that embeds a context c verbatim into an instruction's `context` field. Then, for every $c \in C$:

$$\phi(t(c)) = \phi(c)$$

Proof: By the definition of t, the extracted instruction carries a verbatim copy of the source context; since ϕ is defined over context contents and the copy is content-identical, $\phi(t(c)) = \phi(c)$ holds by construction—a *design invariant of the extraction pipeline*

(the implementation copies rather than re-encodes). At runtime the shipped checks are constraint-based and bracket the transition: `AgenticTransformer.validate_context` checks the registered context constraints pre-dispatch, and `verify_context_preservation(before, after)` re-checks them pairwise across the executed transition and reports violations (mechanics in Section 3.2.3); neither performs structural validation of the carried context. Extraction-time preservation is discharged by the copy construction; preservation across an executed transition is *monitored, not guaranteed*, by the runtime checks. ■

3.2.3 Cross-model Compatibility. Cross-model *protocol* compatibility—not behavioral equivalence of models—is achieved through an abstract interface layer (provider-independent instruction representation with provider-specific adapters):

```
Interface ModelAdapter {
    translate: AgentInstruction → ModelSpecificCall      // shipped:
    ↪ translate_instruction
    normalize: ModelSpecificResponse → AgentResponse    // shipped:
    ↪ normalize_response
    validateContext: Context → Boolean                   // shipped:
    ↪ AgenticTransformer.validate_context
}
```

All three operations ship: `translate` is `ModelAdapter.translate_instruction` (with registered type transformations); `normalize` is `ModelAdapter.normalize_response`, folding the two dominant wire shapes—Anthropic (typed `content` blocks, `usage.input_tokens/output_tokens`) and OpenAI (`choices[0].message`, `usage.prompt_tokens/completion_tokens`)—into a canonical `AgentResponse` envelope (content, tool calls, usage, model, raw), so downstream code never branches on provider syntax; and `validateContext` is `AgenticTransformer.validate_context`, the pre-execution complement of the shipped post-hoc `verify_context_preservation(before, after)`—it evaluates the registered context constraints against a single state before dispatch, applying two-argument predicates reflexively (v, v) (single-argument ones directly) and leaving constraints over absent keys vacuously satisfied, so key *deletion* goes unreported—a blind spot analogous to non-reflexive constraints (e.g. strict growth), which are not detectable pre-execution.

Motivated by cross-model function-calling studies [17] and open protocol efforts [18]—providers differ in surface syntax but can agree on *protocol shape*—the requirement is a **protocol-level contract on the adapter layer**. For $\mathbb{M}$ the set of provider adapters, each $m \in \mathbb{M}$ provides a request normalizer $\text{norm}_{in}(m)$ and a response normalizer $\text{norm}_{out}(m)$ such that:

$$\forall m \in \mathbb{M} : \text{norm}_{in}(m), \text{norm}_{out}(m) \text{ are total on the supported request and response classes,}$$

mapping every supported raw request and response into the unified schemas, with declared types preserved and errors classified into a common taxonomy. This is deliberately weaker than semantic-output equivalence—$m_1 = m_2$ would already demand determinism, which no LLM provides—and cross-model *behavioral* comparison is an empirical question at benchmark level (Section 3.5, [17, 29]).

3.3 Trace-Based Instruction Learning

3.3.1 Dialogue Pattern Recognition. Pattern recognition in AgentISA is frequency-based and deterministic (action-signature grouping) rather than grammatical; it is implemented in `agentflow.core.isa.pattern_mining` and `agentflow.core.isa.instruction_learning`. To prevent over-reading: the subsystem is a **discovery-and-replay pipeline, not an autonomous learning loop**. Its five stages are (i) trace-derived pattern mining (this section), (ii) dialogue-derived template extraction (§3.3.2—templates carry no recorded instruction body and raise `NotImplementedError` if executed), (iii) candidate validation (support and success-rate thresholds), (iv) *developer-controlled* promotion (mining never edits the optimizer's rule set; a human decides which candidates become instructions), and (v) composite replay (a promoted instruction re-runs its recorded subsequence on the same inputs; it does not generalize to unseen parameter values). Stages (i)–(iii) and (v) ship; stage (iv) is deliberately manual.

Definition 3.4 (Instruction Sequence Pattern): A pattern p over the instruction alphabet I is a non-empty contiguous sequence $p = (i_1, \ldots, i_k) \in I^k$ together with two frequency statistics:

- **Support**:
$$sup(p) = P(p \text{ occurs as a contiguous subsequence in a randomly drawn trace})$$
- **Confidence**: $conf(p \Rightarrow j) = P(i_{t+1} = j \mid (i_{t-k+1}, \ldots, i_t) = p)$

A pattern is *frequent* if $sup(p) \geq \sigma_{min}$ and *predictive* if $\max_j conf(p \Rightarrow j) \geq \gamma_{min}$, for thresholds $\sigma_{min}, \gamma_{min} \in (0, 1]$.

Pattern recognition proceeds in three stages:

1. **Frequent-sequence mining.** Candidate subsequences are enumerated from execution traces with occurrence counts; successor statistics are accumulated alongside, and sequences below σ_{min} are pruned together with their statistics—so confidence is estimated only over frequent sequences. The output is the set of frequent, predictive patterns.
2. **Behavior grouping.** The shipped `PatternRecognizer` groups dialogues deterministically by *action signature*—the sorted set of system-turn contents—and promotes each sufficiently supported bucket into a pattern. An earlier prototype (TF-IDF embeddings, density-based clustering, k-means) was abandoned for production use: non-deterministic across library versions and degenerate on the short traces agents actually provide (a k-means view over instruction-trace features remains in `pattern_mining`, with fixed `random_state`). Promotions split by source: trace-mined promotions execute by **composite replay**—the learned instruction's `execute` body re-runs the recorded constituent sequence, with its dataflow edges, in mined order against a shared context, merging outputs and returning the keys its `produces_X` post-conditions declare—while dialogue-mined templates, which carry no recorded body, raise `NotImplementedError` rather than silently no-opping. Promotion is deterministic (pattern identity derives from the mined instruction-id sequence), and

replay is plain sequential composition—nothing smarter—so a learned instruction reproduces its recorded subsequence's outputs on the same inputs but does not generalize to unseen parameter values; no automated mining-to-optimization path exists yet (promotion is a developer decision).

3. **Pattern validation.** Each candidate is checked for significance—support above σ_{min} and, for dialogue-derived patterns, an acceptable success rate—before promotion to a candidate instruction.

This design is deliberately statistical: agent traces are short and noisy, and frequency statistics degrade gracefully where grammar induction would overfit. It also composes with the compiler—a frequent, verified pattern is exactly the candidate for promotion to a fused instruction (Section 3.4.4).

3.3.2 Instruction Extraction Pipeline. The shipped extraction process instantiates the Section 3.3.1 pipeline: frequent-sequence mining (pattern detection), deterministic action-signature grouping (instruction formation, carrying the recorded constituent sequence and its dataflow edges), and threshold-gated promotion with the standard optimization passes applied to any resulting stream (optimization and validation).

3.3.3 Instruction Optimization and Verification. Optimization is coupled to verification: where verified-compilation efforts such as [37] verify the transformations of a whole compiler, AgentISA's verification targets a different granularity—the validity of individual instructions and the obligations of individual rewrites:

Definition 3.5 (Instruction Verification): An instruction i is valid if:

$$\forall s \in S \,:\, \mathrm{pre}(i, s) \implies \mathrm{post}(i, \mathrm{exec}(i, s))$$

The shipped checker (`agentflow.core.isa.bounded_checker`) enumerates the candidate input states derived from the declared parameter constraints—exhaustively for finite domains such as `bool` and enum parameters, and as a bounded, deterministically sampled set (cap 256) otherwise—making it *bounded*, not complete. For every pure instruction it filters the state space by well-typedness and declared preconditions, executes the instruction, and evaluates three obligations per surviving state—progress (no raise where the precondition holds), preservation (declared outputs present and well-typed), and the postcondition itself. Side-effectful instructions are never executed and are reported out of scope; their obligation is delegated to authoring-time tests and review. Theorem 3.2 is conditional on this obligation, Theorems 3.4–3.6 carry the separate rewrite obligations of Appendix C.7, and beyond the enumeration bound the condition remains substantive rather than discharged (Appendices B.7 and C.7):

$$\forall s \in S_{bounded} \subseteq S \,:\, \mathrm{pre}(i, s) \implies \mathrm{post}(i, \mathrm{exec}(i, s))$$

where $S_{bounded}$ is the explored subset; completeness is traded for tractability—the standard bounded-exploration compromise, with the operational-semantics treatment of instruction meaning following [36].

The guarantee therefore comes in two explicitly separate layers. **Layer 1 (theory, conditional):** Theorem 3.2 and the type-safety results of Appendix B hold for every program whose instructions satisfy Definition 3.5 *on all of S*, under the total-definedness premise of Appendix B.7—a mathematical statement about a proof obligation, not about the checker. **Layer 2 (implemented verification):** the shipped checker discharges that obligation only on $S_{bounded}$ (exhaustive enumeration for small finite domains; deterministic bounded sampling with cap 256 otherwise) and only for pure instructions; side-effectful instructions are reported out of scope. Success of the bounded check is evidence toward, not a proof of, Layer 1's premise; no claim in this paper equates the two.

Theorem 3.4 (Optimization Soundness): Every atomic rewrite of the pass pipeline—fusion (Appendix C.2's rule, applied by the combining pass), dead-code elimination, and instruction combining (Section 3.4.4)—preserves the denotation of the instruction sequence it transforms: $\mathrm{sem}(r(i)) \equiv \mathrm{sem}(i)$, read as the partial equivalence of C.1.

Proof sketch: By structural induction on instruction composition. The base case shows each atomic rewrite preserves the denotation of a single instruction; the inductive step lifts preservation through each composition operator of Definition 3.6. The full proof, with the per-rewrite case analysis, is in Appendix C.

The shipped checker (`agentflow.core.isa.bounded_checker.verify_instruction`) implements Definition 3.5 on the bounded state space exactly as described above (enumerate, filter by preconditions, execute only pure instructions); it reports violations structurally:

```
def verify_instruction(instruction, pre_condition, post_condition):
    if instruction.side_effects: return skipped(instruction)  # never
    state_space = generate_state_space(instruction)  # bounded: exhaus
    for state in state_space.states:
        if pre_condition(state):
            next_state = execute(instruction, state)
            check(progress, preservation, post_condition, next_state)
    return verdict  # checked_states, exhaustive, violations by kind,
```

3.4 Instruction Organization and Optimization

3.4.1 Instruction Composition. The AgentISA supports hierarchical instruction composition through a formal composition algebra:

Definition 3.6 (Instruction Composition): Given instructions $i_1, i_2 \in I$, we define composition operators:

$$\begin{aligned}
&\text{Sequential} : i_1 \circ i_2 \\
&\text{Parallel} : i_1 \parallel i_2 \\
&\text{Conditional} : i_1 \triangleright i_2 \\
&\text{Iterative} : i_1^*
\end{aligned}$$

Their denotations (sequential execution; parallel interleaving under disjoint state footprints; guarded execution of i_2 on i_1's triggering outcome; zero-or-more iterations of i_1) are fixed in Appendix C.1.

Composition follows algebraic properties. Properties 1 and 3 hold for all instructions; Property 2 is a *conditional* law—purity and pairwise footprint disjointness together do *not* suffice, and idempotence of i_1 must be assumed separately:

1. **Associativity** (all instructions, up to denotation):
$$(i_1 \circ i_2) \circ i_3 = i_1 \circ (i_2 \circ i_3)$$
2. **Distributivity** (conditional on idempotent i_1 and pairwise-disjoint footprints): the law
$$i_1 \parallel (i_2 \circ i_3) = (i_1 \parallel i_2) \circ (i_1 \parallel i_3)$$
holds under two premises that must be stated separately. **(a) Idempotence of** i_1: $\delta_{i_1} \circ \delta_{i_1} = \delta_{i_1}$ on the precondition-satisfying states—i_1's writes must not change the values its own subsequent execution reads (a constant overwrite of declared outputs qualifies; a read-modify-write of a cell it also reads does not). **(b) Pairwise footprint disjointness**: the denotations of i_1, i_2, i_3 act on pairwise disjoint state components (declared outputs distinct, read sets disjoint from the others' writes). Under (a) the second execution of i_1 reproduces the first's writes; under (b) the other instructions cannot perturb i_1's inputs between its two occurrences; disjoint updates commute, so both sides denote the same net transformation.

   Both premises are necessary, and neither follows from the other or from purity. Purity not implying idempotence is the sharper failure: a pure `update(+1)` applied twice increments twice (purity bars *external* side effects, not writes to declared cells). Taking i_1 as such an update and $i_2 = i_3 = \epsilon$ (the identity, empty footprint, so (b) holds vacuously), the two sides execute i_1 once and twice—from $x = 0$ they yield $x = 1$ and $x = 2$; only premise (a) rules this out. Conversely, without (b), a write of i_2 into a cell i_1 reads changes i_1's second execution even when i_1 is idempotent.
3. **Identity**:
$$i \circ \epsilon = \epsilon \circ i = i$$
where ϵ is the empty (identity) instruction.

Remark (implementation consequence): distributivity also fails for side-effectful instructions, for the same double-execution reason. Because the law is doubly conditional, the shipped compiler does not rely on it at all: parallel grouping is gated on purity plus disjoint dependencies and disjoint outputs, `side_effects=True` instructions are excluded from all such rewrites, and no distributivity-based simplification is applied. The law is stated for the algebra's completeness, not as a license for rewrites.

3.4.2 Instruction Pipeline Design. We formalize the instruction pipeline as a directed acyclic graph (DAG):

Definition 3.7 (Pipeline DAG):
$$P = (V, E, \lambda, \delta)$$
where:

- V: Pipeline stages
- E: Dependencies between stages
- λ: Stage latency function
- δ: Data transfer function

The pipeline's stages (FETCH → DECODE → SCHEDULE → EXECUTE → COMMIT, with hazard detection, dependency tracking, and resource allocation maps; listing in Appendix D.11 of the supplementary material) are organized around this DAG.

Theorem 3.5 (Pipeline Correctness): For any valid instruction sequence s and any schedule S satisfying conditions S1-S4 of Appendix C.5—each instruction dispatched exactly once, dependencies respected, dispatch groups pairwise non-interfering, commit in program order—pipeline execution preserves program semantics.

Proof: By induction over the dispatch groups of S in commit order, under the invariant that the architectural state after committing each prefix of groups equals the sequential state after the same instructions; S1–S4 supply exactly-once dispatch, dependency order, group commutativity, and in-order commit (details in Appendix C.5).

3.4.3 Instruction Selection and Scheduling. Instruction selection follows a tree-pattern matching approach:

Definition 3.8 (Selection Pattern):
$$\pi = (\tau, C, \text{cost}, \text{code})$$
where:

- τ: a tree pattern over the IR (unrelated to the type-system symbol T of Definition 3.2)
- C: context constraints on the surrounding state
- cost: Cost function
- code: Generated code

Given an IR tree t to cover, the selection process is formalized as:

$$\text{Select}(t) = \arg\min_{\pi \in \Pi} \{\text{cost}(\pi) \mid \text{matches}(\pi.\tau, t) \wedge \text{satisfies}(t, \pi.C)\}$$

Scheduling is formalized as the classical delay-minimization problem under dependency and capacity constraints (formulation in Appendix E.1 of the supplementary material): minimize the total stall $\sum_i \text{delay}(i)$ subject to $\text{start}(i_2) \geq \text{end}(i_1)$ for every dependency pair and $\sum_{i \in \text{active}(t)} \text{res}(i) \leq \text{Capacity}$ at every time t—the specified, not shipped, scheduler's objective.

3.4.4 Static and Dynamic Optimization. **Static optimization.** Static optimization employs several techniques:

1. **Peephole Optimization**:
$$\text{PH}(i_1, ..., i_n) = \begin{cases} \text{opt}(i_1, ..., i_n) & \text{if pattern exists} \\ i_1, ..., i_n & \text{otherwise} \end{cases}$$
2. **Dead Code Elimination**:
$$\text{Live}(i) = \text{Use}(i) \cup (\text{Live}_{\text{out}}(i) - \text{Def}(i))$$

The two shipped passes approximate this fixpoint at different conservatism levels: the *compiler* pass (formal streams) removes a side-effect-free instruction when no live consumer depends on its id or any declared output, while the *trace optimizer* pass (executable instructions) keeps any instruction declaring at least one output—executable traces lack the formal dependency annotations that make output-consumption analysis exact. Section 5.3 exercises the former; Section 5.4 the latter.

3. **Instruction Combining** (the named candidate rules below are specified and not registered—Section 4.1.3; the shipped optimizer's built-in pairwise compatibility combining is active, and it is this built-in combining that the hot-path trace re-optimization of Section 5.4 applies):

```
Combine Rules {
  // LOAD/STORE/MOVE are textbook mnemonics, illustrative
  // only; they are not shipped opcode names
  LOAD followed by STORE → MOVE
  Multiple CONTEXT_UPDATE → Single UPDATE
  Sequential QUERY → Batch QUERY
}
```

Dynamic optimization. Dynamic optimization in the shipped runtime is **repetition-triggered and threshold-based**; it does not (yet) respond to load or performance signals:

1. **Hot Path Detection** (implemented): the `DynamicOptimizer` keeps a monotone counter per exact instruction sequence (`path_id` = the ordered join of instruction ids); a path is flagged hot when its count exceeds a configurable threshold *and* its most recent recorded success rate is at least a stability bound (0.9 by default). There is no windowing and no load signal:

$$\text{Hot}(p) \;:\Longleftrightarrow\; \text{count}(p) > \theta \wedge \text{success}(p) \geq \alpha$$

2. **Trace Optimization** (implemented): once hot, the cached sequence is re-optimized by the static pass machinery—peephole, dead-code elimination, instruction combining—bounded to a fixed number of passes; the runtime then serves the shrunken form for subsequent executions (demonstrated end-to-end in Section 5.8).

3. **Trace Formation and Adaptive Recompilation** (specified, not implemented): sub-trace extraction from observed frequency profiles, and performance-triggered recompilation (`should_recompile`: execution count above threshold *and* observed performance below expectation) remain design-level. The shipped runtime re-optimizes the whole cached sequence once per recording after the threshold is crossed, with no performance feedback loop.

Two known consequences of the shipped design: because `path_id` requires exact sequence identity, trace diversity (similar-but-not-identical sequences) never accumulates counts and the mechanism stays cold; and hot-path state (sequences, counters, statistics) is unbounded. Both are discussed with their remedies in Section 6.1.

3.4.5 Verification and Analysis. Optimization correctness is argued through:

Theorem 3.6 (Pass-Pipeline Soundness, extending Theorem 3.4): For any optimization transform T composed of the pass pipeline's rewrites,

$$\forall p \in \text{Program} \;:\; \text{sem}(T(p)) \equiv \text{sem}(p)$$

Proof Sketch:

1. Show semantic preservation for each atomic transformation (the atomic-rewrite lemma, Appendix C.2)
2. Prove composition of transformations preserves semantics (the compositionality lemma, Appendix C.3)
3. Note that hazard-free pipeline execution is *not* part of this theorem—it is Theorem 3.5 (Appendix C.5), invoked separately; this theorem's contribution is the rewrite layer (steps 1–2).

Performance Model (specified; the benchmarks of Section 5 measure latency and token counts directly and do not evaluate it):

$$\text{Perf}(p) = \sum_{i \in p} (\text{base_cost}(i) + \text{overhead}(i) - \text{optimization_gain}(i))$$

where optimization_gain is calculated as:

$$\text{optimization_gain}(i) = \sum_{o \in \text{Opts}} \text{benefit}(o, i) \times \text{application_rate}(o, i)$$

3.5 Capability Assessment

Agents are heterogeneous: an agent may be strong at reasoning but weak at tool use, or vice versa. To make such differences explicit—and to give the multi-agent coordinator of Section 5.6 a principled basis for agent selection—AgentISA includes a capability assessment subsystem, implemented as `CapabilityProfile` in `agentflow.core.isa.capability`.

Definition 3.9 (Capability Profile): A capability profile of an agent a maps each capability domain D (the shipped enum carries eight domains: reasoning, memory, generation, optimization, specialization, coordination, security, robustness) to a score–weight pair:

$$\text{Cap}(a) = \{(\text{dom}_k, s_k, w_k) \mid \text{dom}_k \in D\}$$

where each domain carries a score $s_k \in [0, 1]$ and a weight $w_k \geq 0$ reflecting the domain's importance in the current deployment context.

Implementation status. The aggregation and selection mechanisms ship (`CapabilityProfile`, `get_overall_score`, and the coordinator's threshold selection) and are functionally tested in Section 5.6; the instruction-level metric extraction is a specified stub—the seventeen `_measure_*` functions return 0.0—so `assess_capabilities` has no production call site yet, and Section 5.6's tests inject `DomainScore` values directly. The pipeline's *mechanics* are pinned by unit tests (`tests/core/isa/test_capability_pipeline.py`): end-to-end assessment over a real instruction list produces well-formed results for all eight domains with stub-zero scores; the domain aggregation is the confidence-weighted mean with the weight deliberately absent (guarding against a double-weighting defect observed during development—an earlier form folded the domain weight into s_k as well, capping an assessment

below the coordinator's 0.7 threshold); the top-level weighted mean applies weights exactly once.

Assessment proceeds in three steps:

1. **Domain metric computation.** For each domain, `assess_capabilities` is *specified* to compute a set of `CapabilityMetric` values from the agent's instruction set—e.g., for the reasoning domain, metrics derived from the prevalence and complexity of reasoning-type instructions. As stated above, this extraction is a specified stub in the shipped code, so exercised assessments derive domain scores from injected metrics.

2. **Domain aggregation.** Metrics are aggregated into a per-domain `DomainScore` as a confidence-weighted mean, matching the shipped `_compute_weighted_score`:

$$s_k = \frac{\sum_{m \in M_k} v_m \cdot \mathrm{conf}_m}{\sum_{m \in M_k} \mathrm{conf}_m}$$

where M_k is the metric set of domain k, v_m the metric value, and conf_m its confidence; an empty metric set scores 0, and zero total confidence degenerates to the unweighted mean. The domain weight w_k is deliberately absent here—it enters only once, at the top level (an earlier shipped form folded it into s_k as well, double-counting it; Section 5.6's tests were unaffected because they inject `DomainScore` values directly).

3. **Overall score.** `get_overall_score` computes the weighted mean across assessed domains:

$$S(a) = \frac{\sum_k w_k \, s_k}{\sum_k w_k}$$

with $S(a)$ falling back to the unweighted mean when all weights are zero.

Three properties of this design deserve emphasis. First, it is **static rather than live-probing**: assessment is designed to be cheap, deterministic, and side-effect-free, though the shipped metric extraction is the stub disclosed above and exercised scores are injected. Second, it is **compositional**: because scores summarize instructions, fusing or eliminating instructions during compilation (Section 3.4) changes the profile predictably, and a production deployment would re-assess after optimization (the shipped coordinator does not yet do so). Third, it directly serves coordination: the agent-selection predicate in Section 5.6 chooses the highest-scoring agent above a capability threshold for each task, subject to the security policies of the `SecurityMonitor`—with the tests injecting scores directly, as disclosed above.

We state the scope of this subsystem honestly: `CapabilityProfile` assesses *declared instruction-level capability*, which correlates with—but does not measure—end-to-end task performance. Cross-model behavioral comparison at benchmark level is the domain of evaluation suites such as BFCL [17] and AgentBench [29]; the two views are complementary.

4 Implementation

This section describes the implementation status of Section 3's components, with emphasis on the compiler infrastructure and optimization techniques of Section 3.4.

4.1 Compiler Infrastructure

4.1.1 Instruction Selection Implementation. We *specify* the tree-pattern matching approach of Section 3.4.3 (Definition 3.8) as dynamic programming over the IR tree, in the classical manner of bottom-up instruction selection [33]. **The shipped compiler does not yet implement this selector**: its current selection step is a heuristic matcher—name-overlap scoring over the instruction catalogue—whose compact listing is deferred to the source (`agentflow.core.isa.instruction_selector`); the DP listing below is the *specified* optimal selector, and none of the benchmark numbers in Section 5 depend on tree-pattern selection. The specified algorithm—the MATCH-TREE dynamic program over the IR tree—is listed in Appendix D.1.

Theorem 4.1 (Selection Optimality, specified algorithm): Given a set of context-local patterns P and an IR tree T, the MATCH-TREE algorithm computes the minimal covering cost; the covering itself is read off the `selected` table afterwards, by walking from the root and skipping subtrees spanned by the chosen pattern.

Proof sketch: by induction on tree height; the recurrence sums only over children not covered by the candidate pattern, so minima compose (full argument, with the context-locality side condition, in Appendix D.1). This theorem is a property of the specified algorithm, not of the shipped heuristic selector; no optimality claim is made for the implementation.

4.1.2 Pipeline Scheduling Implementation. We *specify* the pipeline scheduler following the instruction-scheduling formulation of Section 3.4.3 (Deps and resource capacity) and the correctness conditions of Theorem 3.5 (Section 3.4.2), with hazard detection and resource-constrained issue in the style of classical pipeline scheduling [34]. **This cycle-accurate scheduler is design-level**: the executed runtime path is the sequential and parallel-grouped dispatch produced by the compiler's grouping pass (Section 4.1.3); no benchmark in Section 5 exercises a cycle-accurate schedule. The specified scheduler is listed in Appendix D.2.

Schedule verification checks the dependency-ordering condition (S2) of the formal model (Theorem 3.5, Appendix C.5), with the edge convention of Section 3.4.3—$(i_1, i_2) \in \mathrm{Deps}$ means i_1 must finish before i_2 starts. The check as written,

$$\forall (i_1, i_2) \in \mathrm{Deps} \; : \; \mathrm{slot}(i_2) > \mathrm{slot}(i_1),$$

is equivalent to $\mathit{start}(i_2) \geq \mathit{end}(i_1)$ only under the **unit-slot model** the shipped scheduler assumes: one slot per cycle, every instruction occupying exactly one slot, so $\mathit{start}(i) = \mathrm{slot}(i) \cdot c$ and $\mathit{end}(i) = (\mathrm{slot}(i) + 1) \cdot c$ for cycle length c. If instructions with multi-cycle latencies were admitted, the slot comparison would have to be replaced by an explicit *start*/*end* comparison (or slots generalized to intervals); the pipeline scheduler that would need this remains the specified component of Section 4.1.2. S1 (exactly-once dispatch)

and S4 (in-order commit) hold by construction of the scheduling loop, and S3 (group non-interference) is enforced by the grouping check.

4.1.3 Static Optimization Implementation. We implement the optimization passes defined in Section 3.4.4. The passes that fire in the benchmarks are fusion (the instruction-combining pass of Section 3.4.4, listed in Appendix D.10 of the supplementary material), dead-code elimination (the shipped two-level design of Section 3.4.4), and parallel grouping (described with its measurement in Section 5.3). The shipped rule set registers no peephole patterns, so every peephole obligation below is currently dormant but stated for completeness:

1. **Peephole Optimization** (shipped class with an empty pattern table; listing in Appendix D.8):

Each peephole rule carries a correctness *obligation*, discharged as part of rule definition rather than by the running compiler:

$$\forall p \in \text{Patterns}, w \in \text{Instructions}^{\text{len}(p)} :$$
$$\text{matches}(p, w) \implies \text{sem}(\text{replace}(p, w)) \equiv \text{sem}(w)$$

As stated above, the shipped rule set is empty, so the equivalence checker is dormant; Section 4.3.1 specifies the symbolic-execution checker a populated rule set would require.

2. **Instruction Combining**: the shipped pairing pass groups adjacent compatible instructions and verifies each merged group; its full listing is in Appendix D.10 of the supplementary material, and in the source module `agentflow.core.isa.instruction_optimization`.

4.2 Runtime System

4.2.1 Dynamic Pipeline Control. Design of the dynamic pipeline controller (**specified, not implemented**—the shipped dynamic machinery is the counter/threshold hot-path detector and the trace re-optimization described in Section 3.4.4 (both discussed in Section 4.2.2; measured in Sections 5.4 and 5.8); a skeleton module exists but is not wired into the runtime; the listing is in Appendix D.3):

Pipeline performance is modeled with the classical pipeline bound—throughput is the reciprocal of the slowest stage's service time:

$$\text{Throughput} = \frac{1}{\max_i \text{ service_time}(\text{stage}_i)}$$

(a model of the specified controller; nothing in Section 5 measures it)

4.2.2 Dynamic Optimization Implementation. The *shipped* mechanism is the counter/threshold detector described in Section 3.4.4: a monotone count per exact `path_id`, crossed against a configurable threshold with a last-recorded-success-rate gate. What the benchmarks measure (Sections 5.4 and 5.8) is this mechanism. The following is the *specified* detector—exponential moving average over a window, frequency normalized by recent execution share, and trace formation from hot-path regions; it is **not yet implemented**. Together with the block-granular trace formation below it addresses workloads with trace diversity; the detector itself remains exact-path-keyed (the limitation of Section 3.4.4—that similar but non-identical sequences never accumulate a count—still applies to it):

1. **Hot Path Detection** (specified; listing in Appendix D.4):

The decision rule of the pseudocode is the threshold predicate IS-PATH-HOT (frequency above θ and stability above the stability threshold); it computes no score. A *ranking variant*—specified, not part of the decision rule—can order candidate paths by a heuristic score blending the already-normalized frequency with recent stability through a static weight β (formula in Appendix E.1); it is a heuristic score, not a conditional probability.

2. **Trace Formation** (specified; listing in Appendix D.5):

Trace validity is *defined* by the conjunction of three properties (H the loop header of the region, tail the last block of the trace); VERIFY-TRACE-PROPERTIES checks them:

$$\begin{aligned}\text{Valid(trace)} \iff\ & \text{Entry(trace) dominated by } H \\ & \wedge\ \text{Exit(trace) post-dominates tail} \\ & \wedge\ \text{NoSideExit(trace) except tail}\end{aligned}$$

4.3 Optimization Verification

4.3.1 Static Analysis. We *specify* verification of the static optimizations from Section 3.4.4 (a symbolic-execution equivalence checker over transformation path pairs). As stated in Section 4.1.3, the shipped peephole table is empty and this obligation is dormant; the soundness theorems (Theorems 3.4–3.6) are conditional on discharging it when rules are introduced (Appendix C.7). The specified checker is listed in Appendix D.6.

The verification follows the formal model:

$$\text{Equiv}(p_1, p_2) \iff \forall \sigma \in \Sigma : [\![p_1]\!]\sigma = [\![p_2]\!]\sigma$$

4.3.2 Dynamic Analysis. The runtime *monitors* optimizations through a single mechanism (no result-correctness oracle is defined at runtime—correctness of LLM-instruction results is not decidable in general and is delegated to the authoring-time verification of Section 3.3.3; the monitor below is a specified, not shipped, component):

1. **Performance Monitoring** (specified; listing in Appendix D.7):

The evaluation protocol of Section 5.1.3 reports per-metric mean ± standard deviation and makes no inferential (hypothesis-test) claims; no aggregate weighted-mean score is computed.

4.3.3 Empirical Results. The implemented optimizations are validated by the microbenchmark suite of Section 5, which drives the real compiler, cache, and optimization-manager code paths against a mock LLM backend. We keep all numbers in Section 5 and state only the mechanism-to-experiment map here:

Table 1: Mechanism-to-experiment map; every measurement is a controlled-setting result, detailed in Section 5.

Mechanism (implementation status)	Experiment
Result caching (implemented)	Mock-backend caching, 3 skew levels (Section 5.2)
Fusion + DCE static passes (implemented, fire in benchmarks)	Static compilation, 4 levels, 3 lengths (Section 5.3)
Hot-path detection and adoption (implemented)	Dynamic trace re-optimization (Section 5.4)
Pass-level attribution (paired + independent)	Ablation (Section 5.5)
Coordination + security	43 functional tests (Section 5.6)
Cache, compilation, hot-path adoption (live gateway)	Real-gateway validations (Sections 5.7, 5.7.1)
Whole-system composition	Four-task characterization (Section 5.8)

All measurements are controlled-setting results on synthetic workloads (simulated 80 ms per call); production end-to-end claims are out of scope, and Section 5.9 states the threats to validity.

5 Empirical Evaluation

We evaluate AgentISA with a reproducible microbenchmark suite (`examples/benchmarks/isa_microbench.py`) and a multi-agent functional test suite, under the scope fixed at the end of Section 4.3: Sections 5.2–5.5 run against the mock backend; Sections 5.7–5.8 extend validation to a live LLM gateway and four system-level tasks; Section 5.9 discusses threats to validity.

5.1 Methodology

5.1.1 Instrumentation: Mock LLM Backend. All experiments route model inference through a `MockLLMBackend` that counts each `infer` call as one simulated LLM API round trip and charges a fixed 80 ms simulated latency per call to a simulated clock. It never sleeps and never contacts a live model, so every "API call" is observable, deterministic given a seed, and reproducible without network access or inference spend. Everything *around* the backend is real: the cache, compiler, and optimizer are the production `agentflow.core.isa` implementations (`InstructionCache`, `InstructionCompiler`, `OptimizationManager`), exercised through the same code paths a deployment uses.

5.1.2 Workloads. Workloads are synthetic and seed-controlled:

- **Caching**: 2,000 tasks from a discrete distribution over 50 instruction templates × 20 parameter values, popularity Zipf-distributed with exponent $s \in \{0.5, 1.0, 1.5\}$ (larger s concentrates popularity on fewer tasks, increasing reuse potential). Three arms share the same instrumentation (Section 5.2): no cache, the `InstructionCache`, and a naive app-level exact-response cache (a plain dict keyed on the request payload).
- **Static compilation**: random sequences of length 50, 200, and 1,000 from `synthetic_formal_sequence` (`examples/benchmarks/isa_microbench.py`; short enough to audit line by line). Each of the first $n-1$ slots is drawn independently: with probability 0.25 a *dead* instruction (side-effect-free, **no declared outputs, hence no consumers anywhere in the sequence**—removable by construction, planted not inferred); with probability 0.55 a *pure* instruction (side-effect-free, one declared output v_i, consumed **only by the final sink**, fusable next to a same-type pure neighbor); otherwise *side-effectful* (always live, never fused or removed). The n-th slot is a side-effectful *final sink* depending on exactly the set of all pure outputs, so every pure instruction has a downstream consumer and no pure output is deleted as dead. The attributes are disjoint and jointly exhaustive per slot; the passes find the planted DCE/fusion targets, so the measured reductions are properties of this workload family, not of arbitrary agent programs.
- **Dynamic optimization**: instruction traces of length 100 recorded for up to 200 iterations per seed, hot-path detection active (threshold 50 recordings); the loop terminates at the trigger—iteration 50 in all seeds—so each seed contributes exactly 51 recordings (the 50 counted plus the trigger).
- **Ablations**: two arms. The primary arm is *paired*: it recompiles the static-compilation sequences (identical seed base 2,000) with fusion and dead-code-elimination toggled individually and jointly—a same-input decomposition of the 34.2% headline. The secondary arm is an *independent* draw, same distribution but seed base 4,000, a fresh-sequence replication (all-passes total 35.0%, within one standard deviation of the headline).

5.1.3 Protocol and Statistics. Each synthetic-workload configuration (Sections 5.2–5.5) is run with 5 random seeds; we report mean ± sample standard deviation. These are **descriptive statistics only**: we make no hypothesis-test claims (no p-values, no significance tests) anywhere in this paper. Five seeds characterize variability across the tested workload instances, not statistical significance or population-level generality. Section 5.2's naive response-cache baseline shares the mock-backend instrumentation; a formal *inferential* comparison against full external systems remains out of scope, and no such claim is made.

5.1.4 Metric Terminology. Several distinct quantities must not be conflated; each is defined once here:

- **Effective-instruction reduction**—fewer *effective* instructions in the compiled program (Sections 5.3/5.5); an instruction-stream property, not execution time.
- **Dispatch-count reduction**—fewer dispatch operations at runtime, including parallel grouping's folding (Section 5.3); a runtime count.
- **Trace shrinkage**—shorter traces after hot-path reoptimization (Section 5.4); the optimized trace's executed-state identity is verified separately there.
- **Cache hit rate / call reduction**—fraction of calls served from the result cache; here the call-reduction ratio equals the hit rate by construction (Section 5.2).
- **Simulated latency saving**—avoided-call counts × the mock's fixed 80 ms (Section 5.2); derived, not measured.

Table 2: Simulated API calls with and without the result cache (2,000 tasks per seed, 5 seeds, mean ± sample standard deviation; reduction equals hit rate by construction; the third, naive response-cache arm is reported in the text below).

Zipf exponent s	Calls (no cache)	Calls (cached)	Reduction = hit rate
0.5	2,000	810.4 ± 5.7	59.5% ± 0.3%
1.0	2,000	642.0 ± 3.8	67.9% ± 0.2%
1.5	2,000	408.0 ± 9.4	79.6% ± 0.5%

- **Measured latency/token saving**—wall-clock time and token counts against the real gateway (Sections 5.7–5.7.1); measured.

For context preservation (Theorem 3.3): *extraction-time context preservation* is the copy invariance of the extraction step; *runtime context validation* is the monitor's pre/post-dispatch constraint checking—different properties, different coverage (Section 3.2.2 discloses the runtime side's deletion blind spot).

5.2 Experiment 1: Result Caching

Research question. How many simulated LLM API calls does AgentISA's result caching eliminate, as a function of workload skew?

We execute 2,000 tasks per seed, with caching disabled and with the `InstructionCache` enabled, measuring simulated API calls at the backend; the reduction ratio and hit rate coincide by construction (every avoided call is exactly one cache hit).

Cache-key contract. The shipped result cache keys an entry on the (instruction, params) pair—deliberately *not* on model identity/version, prompt-template version, tool schema, decoding parameters, or ambient context. Soundness requires those factors held fixed, which every benchmark here guarantees by construction (single alias, fixed templates and decoding, fresh cache per arm); Section 5.7.1's classifier-keyed step hashes exactly the input dictionary its prompt is built from, under the same discipline. A deployment changing any of these factors must invalidate or namespace the cache (the production cache isolates by TTL, LRU capacity, and never-cached failures but does not detect semantic-configuration drift); cache validity across *varying* model versions or prompts is future work, not exercised by any number in this paper.

Naive response-cache baseline. A third arm runs an app-level *response* cache keyed on the exact request payload string—what an exact-match cache in front of the backend (GPTCache's exact mode, or an HTTP response cache) keys on—implemented as a plain dict with no AgentISA machinery. Across the three skews it issues exactly the same call counts as the table's cached arm: **parity with the `InstructionCache` to the digit**, because the (instruction, params) key is already canonical on this workload. Parity is the finding, stated plainly: at the level of call reduction the instruction-level cache adds nothing over exact-response caching; what it adds is the compile-time transformations of Sections 5.3 and 5.5—fusion and dead-code elimination, reducing the program before any request is issued, which a response cache cannot express because it sees request strings, not instruction semantics. Semantic/embedding caching (GPTCache's approximate mode [46]) remains out of scope: no head-to-head against approximate matching is claimed.

Table 3: Effective-instruction-count reduction by optimization level and sequence length (5 seeds, mean ± sample standard deviation; planted dead 0.25 / pure 0.55 workload).

Sequence length	Level	Effective-instruction reduction	Dispatch count
50	NONE	0% ± 0.0	50.0 ± 0.0
50	BASIC	32.8% ± 4.1%	33.6 ± 2.1
50	ADVANCED	32.8% ± 4.1%	19.0 ± 4.1
50	AGGRESSIVE	32.8% ± 4.1%	19.0 ± 4.1
200	NONE	0% ± 0.0	200.0 ± 0.0
200	BASIC	34.2% ± 1.8%	131.6 ± 3.6
200	ADVANCED	34.2% ± 1.8%	74.0 ± 6.3
200	AGGRESSIVE	34.2% ± 1.8%	74.0 ± 6.3
1,000	NONE	0% ± 0.0	1,000.0 ± 0.0
1,000	BASIC	35.4% ± 1.6%	646.0 ± 16.4
1,000	ADVANCED	35.4% ± 1.6%	344.6 ± 31.6
1,000	AGGRESSIVE	35.4% ± 1.6%	344.6 ± 31.6

Interpretation. The reduction grows monotonically with skew, 59.5% at $s = 0.5$ to 79.6% at $s = 1.5$: the savings' upper bound is the probability mass of repeated tasks, which the Zipf exponent directly controls. In simulated latency the savings are the avoided-call counts × 80 ms ($s = 0.5/1.0/1.5$: 95,168 ± 455 / 108,640 ± 305 / 127,360 ± 750 ms)—derived, not separately measured.

Two caveats keep this honest. First, these are *simulated* calls against a mock backend: the reduction is exact for this workload family, but real deployments face cache-invalidation, prompt-context, and embedding-variability effects the mock abstracts away. Second, the absolute number depends on reuse structure; workloads without repetition would see zero benefit. The experiment establishes the *mechanism and its skew sensitivity*, not a universal constant.

5.3 Experiment 2: Static Compilation

Research question. How much does the compiler reduce instruction sequences, and what do the different optimization levels contribute?

We compile random sequences at four optimization levels and measure two quantities, kept explicit because they are easy to conflate: **effective instruction count**—instructions remaining after fusion and dead-code elimination, a program-size measure—and **dispatch count**—independent dispatch operations the runtime issues, where parallel grouping can fold several independent instructions into one dispatch *without changing the effective instruction count.*

Interpretation. Three observations:

1. **Effective instruction count reduction is level-stable**: BASIC, ADVANCED, and AGGRESSIVE produce identical

reductions (34.2% ± 1.8% at length 200). BASIC's fusion and DCE already capture the available elimination gains; higher levels add *scheduling* improvements (dispatch folding), not elimination—ADVANCED and AGGRESSIVE coincide here, their extra passes finding no further targets on these sequences, differing only on streams carrying combinable patterns. We report this as-is rather than claim monotonic gains.

2. **Dispatch count drops further with parallel grouping**: at length 200, dispatch falls from 131.6 (BASIC) to 74.0 (ADVANCED) with effective-instruction reduction unchanged—adjacent side-effect-free instructions with disjoint dependencies and outputs grouped into shared dispatches. The 74.0 figure is a scheduling reduction, not a further program-size reduction. (An earlier implementation folded more aggressively—44.4 at length 200—but could place side-effectful instructions into shared groups whose non-interference was never established; the sounder grouping here folds less, and Appendix C.5's dispatch-folding corollary now matches the implementation exactly.)
3. **Gains are stable across sequence lengths**: 32.8%, 34.2%, 35.4% at lengths 50, 200, 1,000 respectively, tightest variance at length 1,000 (± 1.6%); length does not materially change the opportunity structure.

Effective-instruction reduction is again a *program-size* metric: it translates to simulated API-call savings only where eliminated or fused instructions carry model inference; the caching experiment measures the complementary reuse effect.

5.4 Experiment 3: Dynamic Hot-Path Optimization

Research question. Does the dynamic optimizer detect and specialize repeatedly executed traces?

We record an instruction trace of length 100 for up to 200 iterations per seed with hot-path detection active (Section 5.1.2). Once a path is flagged hot, the runtime forms an optimized trace; **this experiment measures that trace's length and re-executes it**—the replay verification below checks that it reproduces the original trace's final state exactly. Whether a hot-path-optimized stream preserves a task-level artifact is established separately, on realistic streams, by Section 5.8 (byte-identical output).

Across seeds, hot-path optimization shrinks the trace by **65.8% ± 4.6%**; the seed-to-seed spread (59.0%–70.0%) reflects which subsequences each synthetic trace makes hot, the effect present in all five seeds. As with the static experiments, this is trace-length reduction on synthetic traces—workload-controlled evidence that the detection-formation-reoptimization loop functions, not a wall-clock speedup claim.

Replay verification. Each seed's optimized trace *is* re-executed—every instruction reads its declared dependencies from a threaded state dict and returns deterministic bindings for its declared outputs (side-effectful instructions leave per-id markers)—and its final state is compared with the original's, executed the same way.

Table 4: Hot-path trace shrinkage per seed (length-100 traces, detection threshold 50 recordings). The replay column records the executed-state identity check: the optimized trace, re-executed against an independent state, reproduces the original trace's final state exactly in every seed.

Seed	Original length	Optimized length	Shrink ratio	Replay state identical
0	100	30	70.0%	yes
1	100	41	59.0%	yes
2	100	30	70.0%	yes
3	100	36	64.0%	yes
4	100	34	66.0%	yes

Table 5: Paired per-pass decomposition on the Section 5.3 sequences (length 200, 5 seeds, identical seed base 2,000); the fusion + DCE row reproduces the Section 5.3 headline exactly because the inputs are identical.

Configuration	Instruction-count reduction
NONE	0.0% ± 0.0%
fusion only	21.6% ± 1.2%
DCE only	24.5% ± 2.6%
fusion + DCE	34.2% ± 1.8%

The states are identical in all five seeds (5/5; recorded per seed as `replay_state_identical` in the results file), so the 65.8% shrinkage carries an executed-equality check at trace level—on the deterministic mock, not against a live model; this is the same style of check as Section 5.8's task-level byte-identity, applied to the synthetic traces. Live-model behavior remains untested—no mock experiment can address it.

5.5 Ablation: Per-pass Contribution

To attribute the static gains we run the two ablations of Section 5.1.2. The primary one is **paired**: it compiles the *same* sequences as Section 5.3 (identical seed base 2,000 and generator) with fusion and DCE toggled individually and jointly, so per-configuration differences are same-input paired differences—an exact decomposition of the headline, not a replication:

The fusion+DCE row is 34.2% ± 1.8%—the headline itself, confirming that these two passes account for the entire static gain at length 200. The **paired interaction**, computed per seed as (fusion+DCE − NONE) − (fusion − NONE) − (DCE − NONE)—34.2 − 21.6 − 24.5 = −11.9 points—is −11.9% ± 2.0% (per-seed −14.5/−9.5/−13.0/−12.0/−10.5 points): the passes overlap—dead slots break fusion chains and fused pairs absorb neighbors DCE would otherwise have removed individually—so the sum of the single-pass gains (46.1 points) overstates the joint effect by 11.9 points.

The secondary ablation is the earlier **independent-draw replication** (seed base 4,000, same distribution): fusion alone 21.9% ± 3.5%, DCE alone 24.1% ± 3.3%, fusion+DCE and all passes 35.0% ± 3.7%—the paired structure reproduces on fresh sequences (single-pass

shares agree within one standard deviation), so the decomposition is not an artifact of one draw. "All passes" equals fusion+DCE here, consistent with Section 5.3's level stability: no additional pass contributes measurable reduction, and we report that absence rather than dress the higher levels in unearned credit. Because dead instructions are planted at rate 0.25 by construction, DCE's per-pass share is near-tautological for this family—it measures that the pass finds the planted dead code, precisely its function.

5.6 Multi-Agent Coordination: Functional Validation

The multi-agent layer—`AgentCoordinator`, `SecurityMonitor`, and the collaborative instruction set—is validated functionally by 43 tests, all passing—the coordination-and-security subset of the repository's full suite (861 passing, 14 skipped at run time), which also covers the ISA core, compiler, and optimizer units of Sections 5.2–5.5. This is **qualitative functional validation**: the tests verify the coordination behaviors exist and are correct, not that they are fast; no performance number should be inferred here.

The coordination flow implemented by `AgentCoordinator` (authenticate, authorize, select by capability, plan stages with fallback, execute, record history) is listed as pseudocode in Appendix D.10 of the supplementary material. What the 43 tests verify, grouped by subsystem:

1. **Agent registration and role policies**: registration creates policies; permissions and security requirements differ by role; trust levels are bounded and reward declared specializations.
2. **Task coordination**: coordination produces a solution or records failure; missing authentication/authorization and no-suitable-agents raise; selection respects the capability threshold, the coordinator being the highest-scoring agent.
3. **Execution planning**: stage structure with coordination rules; every stage has differentiated fallback keys, and a fallback executes on stage failure; validators are reasoning-heavy non-coordinator agents.
4. **SecurityMonitor**: valid tasks pass, missing authentication/authorization fails, violations are timestamped, declared data sensitivity is enforced (sensitive categories infer a high sensitivity level, benign ones pass at medium), resource access is open by default and restrictable by configuration.
5. **Collaborative instructions**: barriers wait for the declared participant count and time out on a missing peer; resource allocation rejects over-allocation and serializes concurrent requests; the workflow state machine accepts valid actions and rejects invalid ones and double starts; a research-team scenario runs end to end.

These tests exercise the same production modules used elsewhere and are fully deterministic (no LLM calls). They establish correctness of the coordination machinery under the tested conditions; what they do not establish is scalability under contention, crash recovery, or adversarial inputs beyond the policy checks above—all future work (Section 6).

Table 6: Real-gateway cache behavior against the computed expectation (200 sequential requests, single seed, three Zipf levels, single model; paired cells read no cache -> cached).

Zipf s	Expected hit rate (computed, no backend)	Observed hit rate	Calls	Call reduction	Wall clock (ms)	Wall saved (ms)	Tokens saved
0.5	8.5%	8.5%	200 -> 183	8.5%	472,276 -> 405,754	66,522	1,445
1.0	26.0%	26.0%	200 -> 148	26.0%	455,946 -> 334,597	121,349	4,420
1.5	45.5%	45.5%	200 -> 109	45.5%	450,330 -> 244,316	206,014	7,735

5.7 Real-LLM End-to-End Validation of Result Caching

Research question. Does the caching mechanism behave identically when the "model" is a real LLM behind a live gateway, and how much *measured* wall-clock time and token usage does it save?

Setup. We replace the mock backend with a real model behind a local Anthropic-protocol gateway (TLS with an internal CA; alias `claude-glm-5.3-flash`, a hosted reasoning-capable GLM model; `max_tokens=64`; sequential httpx, 90 s timeout, three bounded retries—none occurred). Every other component is the production code path: the same `InstructionCache` protocol, instruction objects, and workload generator, with Section 5.2's instruction pool (50 templates) and parameter space (20 values).

Scale. With each round trip costing ~2.2 s, scale drops from 2,000 tasks × 5 seeds to **200 tasks × 1 seed** per Zipf level (s in {0.5, 1.0, 1.5}): 1,040 measured round trips (plus one preflight) in 39.4 minutes. The seed pins the task sequence exactly, so the workload is the one the expectation below is computed from.

An exactness property. A cache hit depends only on the (instruction, params) pairs seen so far—not on what the model returns. The production `InstructionCache` is LRU with a TTL, so in general a hit also depends on wall-clock time and capacity; under this benchmark none of that can bind: `max_size` = 100,000 (at most 200 distinct keys per arm), `ttl` = 3,600 s (every cache-enabled arm completes in under seven minutes; the no-cache arms run to 7.9 minutes and never consult a cache), a fresh cache per arm, failed calls never cached. Hit/miss decisions are therefore a pure function of the (instruction, params) sequence, *computable without any backend*; if the mechanism is correct the real run must reproduce that sequence exactly—and at the recorded granularity (per-arm hit rates and call counts) it does:

Per-call latency was stable across arms (p50 2.13–2.21 s, p95 3.06–3.54 s). Token savings are gateway-reported `usage` differences: every avoided call saves its input tokens plus the tokens the reasoning model generates before responding (output tokens were the binding constraint of `max_tokens=64` in every call).

Interpretation. First, the mechanism transfers unchanged: observed hit rates match the computed expectation exactly at every

skew level, so Section 5.2's synthetic results measure the cache, not a mock artifact. Second, avoided calls convert into measured time and tokens—14.1%, 26.6%, and 45.8% of the no-cache arm's wall clock at s = 0.5, 1.0, 1.5—tracking but not identical to the call reduction, since arms ran sequentially and per-call latency drifted slightly (the exact quantity is the call reduction; wall and token figures are measurements, not derivations). Third, the *absolute* hit rates here (8.5–45.5%) are lower than Section 5.2's (59.5–79.6%) purely because repetition accumulates with task count: at N = 200 these are the exact computed values for this workload family, and at N = 2,000 the same computation yields the Section 5.2 numbers—a property of the workload, not the backend.

Scope of the claim. This experiment validates the caching mechanism end-to-end—one gateway, one model, one seed, N = 200, sequential requests, no streaming—recording per-arm aggregate hit rates and call counts; per-position identity follows from the deterministic cache decisions above but is not separately logged, and nothing here is a throughput, availability, or dollar-cost claim. Sections 5.3–5.5 are structural, never dispatching to the backend; Section 5.7.1 extends the real-gateway validation to them via a cache-mediated protocol, their validity otherwise resting on the code-path and ablation arguments there.

5.7.1 Real-Gateway Validation of Compilation and Hot-Path Optimization. **Research question.** Do static compilation and closed-loop hot-path adoption preserve the observable semantics of an instruction stream when the stream's LLM step is served by a real model—and what do the mechanisms measurably save on a live gateway?

Setup, arms, scale. A dedicated harness (`examples/paper_real_llm_compiler_test.py`; tests in `tests/system/test_real_llm_compiler_test.py`) reuses Section 5.7's gateway configuration (same base URL, internal-CA TLS, alias `claude-glm-5.3-flash`, sequential httpx, 90 s timeout, three bounded retries) but raises `max_tokens` to 512 (Section 5.7 used 64) so completions dominate the priced comparison; it runs identically online or offline against a deterministic same-protocol mock (call counting, template-accounted tokens, constant 80 ms simulated latency). The task is Section 5.8's service-log triage shape: a seven-instruction stream with one cacheable LLM-type step (`classify_signature`, one model call per distinct incident-signature batch) and the same planted dead guard (`check_format`), over 50 realizations × 1 seed × Zipf s = 1.0 from a fixed universe of 40 task instances—22 distinct signature batches, so the entire run is 23 gateway round trips (one preflight plus 22 live calls, all by arm (a)) in 4.4 minutes, a bounded validation run, not the 200 × 3-skew sweep of Section 5.7. Arm (a) is the unoptimized authored stream, run first, cold, performing every live round trip; arm (b) drives the *same* stream through the production machinery—`optimize_hot_path` adoption plus static `InstructionCompiler` at NONE vs AGGRESSIVE with the dataflow-preservation assertion—then executes the same draws against arm (a)'s warmed cache. The classify step's cache key is the sha256 of exactly the input dictionary its prompt is built from, so arm (b) issuing *zero* new gateway calls proves it computed identical keys—hence identical prompts and consumed values—and its artifacts must come out byte-for-byte equal; any perturbation of the LLM step's inputs would surface as a cache miss and/or artifact divergence.

Results. All checks pass. Arm (a) made exactly the 22 first-occurrence calls the backend-free computation predicts and served the other 28 realizations from the result cache (hit rate 56.0%), consuming 4,027 prompt and 11,264 gateway-reported completion tokens in 260.2 s of measured call latency. Arm (b)—compiled, hot-path-adopted—made zero new gateway calls and reproduced every artifact byte-for-byte. The hot path was adopted at realization 7 (threshold 5; adoption lands two realizations past it because the counter must strictly exceed it and recording follows execution, as in Section 5.8): realizations 1–6 executed the full seven-instruction stream, realizations 7–50 the shrunken six-instruction form with `check_format` absorbed into its successor by fusion—306 instruction executions against 350 in arm (a), a 12.6% executed-instruction reduction attributable to the adopted trace (per-stream shrink 14.3%, 7 → 6; realized saving diluted by the six full-form realizations before adoption; the static compiler alone reproduces the 7 → 6 shrink at AGGRESSIVE, so the two mechanisms' contributions are not separated), reported separately from the cache's call savings. Static compilation of the same stream: NONE 7 → AGGRESSIVE 6 effective instructions, dataflow preserved.

A priced comparison (summary; protocol and table in Appendix G of the supplementary material). Costs are MODELED—computed from a pricing overlay on measured tokens, not observed: the overlay prices the run's token counts under no optimization, AgentISA result caching (first occurrences only—the measured behavior of both arms), and a modeled provider prompt cache (every realization queries; repeat prompts bill input tokens as prompt-cache reads at 0.1x, mirroring Anthropic's published read discount; completions always bill at full price). Two simplifications—omitting Anthropic's 1.25x cache-write premium and 1024-token minimum cacheable prefix—both favor the provider-cache baseline, so the comparison is conservative for us; list prices are parameterized stand-ins ($3/M input, $15/M output) because the gateway's actual pricing is unknown; no dollar-savings claim is made. On this run's token mix the result cache cuts modeled cost by 56.0%—it removes whole calls, completions included—while the modeled provider prompt cache cuts only 3.3%, the reasoning model's completions dominating; on the deterministic mock at paper scale (200 tasks × 1 seed × 3 Zipf levels) the result cache cuts modeled cost by 80.1%, 81.6%, and 85.1% at s = 0.5, 1.0, 1.5, against 28.9–31.4% for the provider prompt cache—there short completions do not dominate, so the input-side discount matters more; the ranking of the two caching strategies is a function of the input/output token mix, not a constant. GPTCache, an external cache layer, is explicitly out of scope; no head-to-head against it is claimed.

What this does and does not show. The byte-identity property is cache-mediated by design: the cached values entered the artifacts once, through arm (a)—values that on this alias are fallback-derived, as explained below—and arm (b) is proven to compute the same keys and consume the same values; this validates the stream transformation, not model determinism, which is not guaranteed

at temperature 0 and is not claimed. On this alias the reasoning model's replies bind at `max_tokens` in every call (as at `max_tokens=64` in Section 5.7, and as a 1,500-token diagnostic still did), so the classify step's severity values come from its deterministic fallback; what is validated against the live model is the mechanism chain (key computation, cache decisions, call and token accounting, artifact identity), not classification quality.

What the real-gateway experiments do and do not establish. Three layers, kept separate: (1) *gateway integration correctness*—requests reach the gateway, the cache governs calls, and call/token statistics are recorded faithfully: **established** by Sections 5.7 and 5.7.1. (2) *Workflow semantic preservation*—the optimized stream computes the same cache keys and consumes the same cached values as the unoptimized one, yielding byte-identical artifacts: **established as a cache-mediated identity** (see above), validating the stream transformation, not model determinism. (3) *Real-model task quality*: **not established here**—the reasoning alias returned results at the token ceiling and the classify step fell back to a deterministic rule, so no statement about live-model output quality is made anywhere in this paper; the abstract, highlights, and conclusion state only layers (1) and (2). Both real-gateway runs share the coverage limits above (one gateway, one model, one seed, sequential, no streaming); multi-model, multi-seed, and real-deployment behavior remain untested.

5.8 Controlled Four-Task System-Level Characterization

The experiments above measure individual mechanisms on synthetic sequences; this section is a *controlled four-task system-level characterization*—a functional demonstration under a fixed closed-loop protocol (same task, 12 realizations, one cacheable step each), not a generalization experiment; the identical 91.7% hit-rate figure below is a property of that protocol, not four independent cache observations. The seed-controlled characterization (`examples/system_task_workload.py`, `tests/system/test_system_task_workload.py`; the original single-task demonstration lives on as `examples/system_task_demo.py`) is built from **four tasks with genuinely different dependency shapes**—linear service-log triage, fan-out csv-etl-anomaly, wide-fan-in report-merge, two-branch regression-diff—each realized end-to-end against sandbox-resident files (12 realizations each, hot-path threshold 5, one planted dead guard and one cacheable LLM-type step per task, the latter backed by a deterministic rule stand-in counting simulated model calls), making the mechanisms' composition—and Section 3.4.4's closed-loop optimization claims—checkable. Per-task instruction streams, the single-task walk-through, and both result tables are in Appendix H of the supplementary material; the headline: every task's guard survives compilation (never dropped outright, the reduction being fusion), the first realization pays all model calls and every subsequent one hits the cache (91.7%, 11 of 12 realizations by construction—one cacheable step, identical seeded draws), hot-path adoption lands at repeat 7 executing the shrunken six-instruction stream, and all reports come out byte-identical (12/12 plus fresh-stack SHA-256 equality); the compile shrink is the only shape-dependent quantity (12.5–25.0%), which is what one wants from an infrastructure claim—the composition behavior is not an artifact of one stream's structure.

The triage task's instruction stream (seven instructions over sandbox-resident files): `ingest_logs` reads six 400-line sandbox log files (side-effectful); `check_syntax` is a defensive format guard that never rejects here (a *planted dead guard*, standing in for real pipelines' belt-and-braces checks); `normalize_entries` parses and normalizes lines; `extract_incidents` lifts them into incident records; `classify_signature` triage-classifies each incident (LLM-type, cacheable, backed by a deterministic call-counting rule stand-in); `aggregate_triage` summarizes by severity; `write_report` writes the report (side-effectful).

Calibrating the synthetic headline. Section 5.4 reports 65.8% trace shrinkage on synthetic traces with planted dead instructions; the four realistic tasks show **12.5–25.0%** (one planted guard in a seven- or eight-instruction stream, plus one additional live-pure fusion pair in csv-etl-anomaly). We state both figures rather than privilege either: 12.5–25.0% reflects the planted dead-check density (one guard per seven- to eight-instruction stream)—a stated density assumption, not observed ordinary-pipeline waste—while the synthetic figure is an upper bound under the skew/planting parameters we control, not a typical-case estimate. Constructing a realistic (non-synthetic) dependency shape also served the audit purpose, exposing two grouping/fusion soundness gaps—side-effectful instructions opening dispatch groups, fused instructions losing their external dependencies—now fixed with regression tests; Section 5.3's dispatch-folding numbers reflect the corrected (more conservative) grouping.

Structural invariance. Every task passes the same structural assertions: the guard is never dropped outright (the eliminated set is empty in all four), the reduction being fusion—in csv-etl-anomaly a second adjacent pure pair also fuses, hence its 25.0% shrink with a single planted guard. The first realization pays all model calls and every subsequent one hits the cache—the 91.7% hit rate is 11 of 12 realizations by construction (one cacheable step, identical seeded draws), so the four tasks share it structurally. Reuse and closed-loop adoption are thus invariant across the four dependency shapes under the fixed protocol, while the guard/fusion shrink varies with stream structure (12.5–25.0%)—the compile shrink is the only shape-dependent quantity; the characterization's value over the single-task demonstration is evidence the composition behavior is not an artifact of one stream's structure.

5.9 Limitations and Threats to Validity

We enumerate the limitations of the evaluation explicitly:

1. **Mock backend, not a live LLM.** The mock charges a constant 80 ms per simulated call and returns deterministic responses; beyond counting, it covers Section 5.4's replay verification (final-state identity, 5/5 seeds) and Section 5.2's naive-cache baseline (same instrumentation). The real-gateway experiments cover the cache reproducing the computed hit/miss statistics and converting avoided calls into measured time and tokens (Section 5.7), and the compiled, hot-path-adopted stream issuing zero new gateway calls with byte-identical

artifacts (Section 5.7.1)—a cache-mediated identity validating the stream transformation, not model determinism. Still not covered: multi-model and multi-seed behavior, real deployment, caching under model nondeterminism *across prompt variations*, streaming responses, provider rate limits, and live-model task quality (the four-task classification steps are deterministic rule-based stand-ins, and the 5.7.1 gateway run fell back to the deterministic classifier).

2. **Synthetic workloads.** Zipf-distributed synthetic tasks are a reasonable model of skewed agent workloads, but a model; Section 5.8's task-level characterization only partially addresses this. The cache-reduction figures (59.5–79.6%) are exact for these workload families and should be read as *skew-sensitivity curves*, not universal savings; uniformly novel tasks would see near-zero benefit. Section 5.8's four-shape characterization calibrates the 65.8% synthetic headline downward with its 12.5–25.0% per-task compile shrink and shows uniform reuse and closed-loop adoption under the fixed protocol (12 seeded realizations, threshold 5)—but the four tasks remain seed-controlled, hand-constructed streams, not a production workload study.

3. **No production end-to-end evaluation.** We have not deployed AgentISA in a production setting; consequently this paper contains no latency-under-load, throughput, availability, or dollar-cost measurements, and we make no such claims.

4. **No real cost data (token usage measured; pricing MODELED on measured tokens).** All latency figures in Sections 5.2–5.5 are simulated (avoided calls × 80 ms); Section 5.7 measures real wall-clock time and gateway-reported tokens at reduced scale, and Section 5.7.1 prices its own measured tokens (the 22-call reference run) under three modes—no optimization, AgentISA result caching, and a modeled provider prompt cache with a parameterized cache-read discount (default 0.1, after Anthropic's published prompt-caching economics). The costs are MODELED-on-measured: tokens are measurements, prices are configurable list-price stand-ins (the gateway's actual pricing is unknown), and no dollar-savings claim is made. Section 5.2's baseline is an *exact-match* response cache (parity reported); head-to-head against *approximate* (semantic/embedding) caching (GPTCache's approximate mode—explicitly out of scope) and a priced deployment study under real pricing remain future work.

5. **Program-size metrics, not wall-clock time.** Effective-instruction and dispatch-count reduction are structural metrics: they correlate with—but do not equal—wall-clock improvements, which depend on per-instruction cost, available parallelism, and dispatch overhead.

6. **Descriptive statistics only.** Synthetic configurations use five seeds (Section 5.1.3); the real-gateway and system-level runs are single-seed by design. No hypothesis tests, confidence intervals, or significance claims are made; standard deviations indicate stability across seeds, nothing more.

7. **Functional, not performance, validation of coordination.** The 43 multi-agent tests verify behavioral correctness under deterministic conditions, not throughput or scalability.

Despite these limitations, the evaluation achieves its purpose: it demonstrates, under fully disclosed and reproducible conditions, that each optimization mechanism is implemented, active, and produces the claimed structural effects on workloads designed to expose them.

6 Discussion

6.1 System Limitations and Challenges

1. **Instruction-set coverage**: complex reasoning patterns are not yet naturally expressible (domain-specific extensions are likely needed), and maintaining consistency of pre/post-conditions and typing rules across extensions remains an open engineering challenge.

2. **Evaluation scope**: as detailed in Section 5.9, the quantitative evidence is primarily microbenchmark-based (Sections 5.2–5.5), supplemented by real-gateway spot checks (Sections 5.7 and 5.7.1) and a seed-controlled four-task characterization (Section 5.8); production end-to-end evaluation—including real provider pricing, rate limits, and failure modes—is required before any deployment claim.

3. **Performance bottlenecks**: cache memory consumption under large instruction universes; synchronization overhead in barrier-coordinated multi-agent execution; recompilation cost when hot-path thresholds are misconfigured; the shipped hot-path detector keys on exact sequence identity, so trace diversity never accumulates counts (the EMA-based windowed detector specified in Section 4.2.2 is the designed remedy); hot-path state is currently unbounded—production use needs an eviction policy and a cap on tracked paths.

4. **Integration challenges**: adapter maintenance as provider function-calling surfaces evolve; version management of learned instructions across LLM updates.

6.2 Security and Privacy Considerations

AgentISA's implemented security layer is the `SecurityMonitor` of Section 5.6—task-level authentication, role-based authorization, data-sensitivity classification, configurable resource-access policies—consulted by the coordinator before every assignment, with violations timestamped. One shipped default deserves an explicit call-out: **resource access is open by default** and restrictable by configuration—a fail-open stance shifting the security burden onto the policy writer; a hardened deployment should invert it to fail-closed (deny on unregistered resources, unknown sensitivity, or incomplete policy), which the current checks do not do. The 43 functional tests validate the implemented checks under configured policies, not the default posture; nothing in this paper constitutes a security certification. Beyond what is implemented, hardening considerations—not claims about the current system: (i) **data privacy**—end-to-end data-flow tracking and privacy-preserving computation remain future work (Section 6.3.6); (ii) **system security**—secure multi-agent communication channels

and audit-log integrity are not yet addressed; (iii) **compliance**—regulated deployments require externally audited controls; the system provides enforcement hooks, not certification.

6.3 Future Research Directions

Readers pressed for time need only the first tier. **Directly motivated by this paper's exposed gaps:** 6.3.1 (unifying the formal semantics and discharging the conditional theorems—responding to Sections 3.3.3 and C.7's specified-vs-shipped split), 6.3.4 (calibrating capability scores against end-to-end performance—responding to Section 3.5's stub disclosure), and the real-workload evaluation named in 6.3.3's first item (responding to Section 5.9). **Longer-term, with no current implementation link:** 6.3.2 (self-modifying and context-aware instruction sets—formulas are sketches, as marked inline), 6.3.5 (emergence metrics, cross-agent transfer), and the privacy composition of 6.3.6.

6.3.1 Theoretical Foundations.

1. **Formal semantics of agent instructions**: consolidating Section 3.1 and Appendix B into one standalone semantics—instruction language, state space, reduction rules, type system—is the foundational next step; nothing in it is new machinery, and the composition is what a journal-length treatment would spell out.
2. **Compositional reasoning**: a parallel-composition rule whose disjointness side condition mirrors the $\|$-typing rule of Definition 3.6—without it the conjunction of preconditions does not guarantee either component's precondition during interleaving—would enable modular verification, parallel composition, and temporal property preservation (rule stated in Appendix E.1 of the supplementary material).
3. **Type theory for agent capabilities**: a capability type discipline with Basic/Composite/Emergent kinds and a composition rule $a \circ b : \text{cap}_1 \otimes \text{cap}_2$ (Appendix E.1).

6.3.2 Advanced Instruction Set Evolution.

1. **Self-modifying instructions**—a formal model for instruction evolution (f is left abstract; the sketch names the inputs, not a concrete update rule):

$$I_{t+1} = f(I_t, \text{Performance}_t, \text{Context}_t)$$

   under semantic-preservation, performance-monotonicity, and safety constraints.
2. **Context-aware optimization**—for each observed context, predict the instruction set's performance against a threshold, and if it falls short, select a best rewrite variant for that context and accumulate it into the set. The programmatic sketch (OPTIMIZE-CONTEXT) is listed in Appendix D.9; its primitives are future work, not implemented machinery.
3. **Cross-model instruction mapping**: a universal mapping framework $\phi_{A \to B} : I_A \to I_B$ with semantic preservation, performance bounds, and capability awareness stated as goals, not delivered guarantees.

6.3.3 Theoretical Performance Bounds.

1. **Asymptotic analysis**: deriving cost bounds per composition operator—when does iterative composition i_1^* admit a closed-form cost model, and under what reuse structure does caching yield amortized $O(1)$ effective cost per task? The microbenchmarks (Section 5) measure the constants empirically; the asymptotic theory remains open.
2. **Resource consumption models**: a calibrated model $R(i) = \alpha C_{cpu}(i) + \beta C_{mem}(i) + \gamma C_{io}(i)$ with platform-specific coefficients fitted against real deployments rather than assumed (Appendix E.1).

6.3.4 Capability Assessment: Extensions. The instruction-level capability assessment subsystem (`CapabilityProfile`, Section 3.5) ships in part—aggregation and the coordinator's threshold selection are implemented and used, while instruction-level metric extraction remains a specified stub. Two extensions remain future work: (i) **benchmark-calibrated metrics**, correlating static, instruction-derived domain scores with end-to-end benchmark performance, in the manner of function-calling evaluations [17] and agent benchmarks [29], so that weights w_k are empirically grounded rather than configured by hand; and (ii) **cross-model comparison matrices**, extending the per-agent profile to a pairwise framework $\Delta(M_1, M_2)$ over capability dimensions, supporting systematic model selection for a given instruction workload.

6.3.5 Emergent Behavior Analysis.

1. **Collective intelligence modeling**: the open question is a formal metric for when multi-agent composition exceeds the sum of its members' capabilities—what emergence would have to mean measurably, rather than an appeal to an undefined φ over unioned capability sets.
2. **Learning transfer mechanisms**: knowledge transfer across agents composes the mining-extraction-validation pipeline of Section 3.3 rather than introducing new machinery—patterns mined from a source agent's traces would be adapted to a target agent's instruction set and re-validated before promotion; the validation obligation transfers unchanged.

6.3.6 Security and Privacy Extensions.

1. **Information flow control**: the classical noninterference property $\text{Secure}(P) \iff \forall s_1, s_2 : L(s_1) = L(s_2) \implies L([\![P]\!]s_1) = L([\![P]\!]s_2)$, where L assigns security levels to states and outputs, stated for terminating runs (divergence and channel leakage out of scope; Appendix E.1).
2. **Privacy-preserving computation**: a direction, not a delivered mechanism—if agent computations over sensitive data were routed through per-step differentially private mechanisms, the overall loss would follow a composition theorem over the per-mechanism (ϵ_i, δ_i) parameters. We deliberately state no bound: which composition theorem applies, and which concrete mechanisms the runtime would expose, are open design choices; any formula written now would be decoration rather than a guarantee. The direction is listed

because the instruction contract is the natural place to carry such annotations once the mechanisms exist.

None of the frameworks above is implemented today; Sections 6.3.3 and 6.3.4 are stated precisely enough to be falsifiable, while the remaining sketches are programmatic, guiding subsequent work from controlled-setting-validated infrastructure toward a deployable, formally verified agent platform.

7 Conclusion

This paper presented AgentISA, an instruction set architecture that treats LLM agent behavior as a first-class compilable artifact. Our contributions are:

1. A formal instruction set with compositional semantics, a type system, and pre/post-conditions, with sanity results as design rationale (Turing completeness as negative certification, Theorem 3.1; pure-core type safety, Theorem 3.2; extraction-time context preservation, Theorem 3.3) and semantic-preservation arguments for the optimization and scheduling transformations (Theorems 3.4–3.6).
2. Agentic Calling, which lifts provider-specific function calling into reusable, provider-independent, cacheable agent instructions.
3. Trace-based instruction learning—frequency mining with deterministic action-signature grouping, developer-controlled promotion, and composite replay.
4. A compiler and runtime applying classical optimizations—fusion, dead-code elimination, scheduling, and dynamic hot-path specialization—to agent instruction streams.
5. A multi-agent coordination layer with capability-based selection and a security monitor enforcing authentication, authorization, and data-sensitivity policies.

The evaluation shows the optimizations are real and measurable under controlled conditions: 59.5–79.6% simulated-call reduction from caching, 34.2% instruction-count reduction from static compilation, 65.8% trace shrinkage from hot-path optimization (Sections 5.2–5.5), real-gateway validations of the cache and the compiled pipeline (Sections 5.7 and 5.7.1, gateway-integration and cache-mediated-identity layers only), a four-task system-level characterization (Section 5.8), and 43 functional tests for the multi-agent layer. These are microbenchmark and controlled-setting results, not production end-to-end measurements; the threats to validity of Section 5.9 stand.

Implementation status of the contributions. For auditability, the five contributions above split by status: *implemented and exercised by the benchmarks*—the instruction set and contracts, Agentic Calling, result caching, fusion/DCE/grouping, hot-path detection and adoption, the coordination layer with injected-score capability selection, and the security monitor (43 tests across both); *implemented with known limits*—the bounded checker (pure instructions, bounded state spaces only), runtime context validation (deletion blind spot disclosed), and pattern mining with composite replay (developer-controlled promotion, no generalization beyond recorded parameters); *specified, not implemented*—tree-pattern instruction selection, cycle-accurate pipeline scheduling, symbolic-equivalence checking, and capability metric extraction. Theorems 3.4–3.6 are conditional arguments for the implemented passes; the specified components carry no verification claim.

AgentISA's longer-term significance is architectural: by giving agent behavior an instruction-level contract, it makes agent systems amenable to the same discipline that transformed hardware and compiler engineering—precise specification, portability across backends, and semantics-preserving optimization. Establishing the corresponding gains on production workloads, and hardening the conditional sanity results and semantic-preservation arguments into mechanically verified implementations, is the immediate next step.

Artifact Availability

The full proofs of Theorems 3.1 and 3.2 and of Theorems 3.4–3.6 (Appendices A, B, and C, referenced throughout), together with the listings of the specified-only algorithmic components (Appendix D), the formal sketches of the future directions (Appendix E), and the exact artifact-regeneration command lines (Appendix F) are provided in the supplementary material, built from this repository as `docs/AgentISA_supplementary.pdf`; the commands are mirrored in the repository's top-level `REPRODUCING.md`.

All artifacts are available in the repository accompanying this submission (anonymized for review). Reproducibility anchors:

- **Version**: every number in Section 5 is generated from the exact repository state whose build produces this PDF; the artifact's `git rev-parse HEAD` at submission time is recorded in `REPRODUCING.md`, alongside the dependency lock file and the Python/system versions used for the reported runs.
- **Clean-environment regeneration**: from a fresh clone, `python -m pytest tests/ -q` reproduces the full-suite count (861 passed, 14 skipped at run time); the Section 5.6 files—`tests/core/isa/test_coordination.py`, `tests/core/instructions/test_agent_coordination.py`, and `tests/system/`—reproduce the functional-test counts (exact command in `REPRODUCING.md`), and the Appendix F command lines regenerate each results table from scratch (including the real-gateway runs, whose configuration—endpoint aliasing, model id, decoding parameters—is documented inline in the harness with a mock-backend fallback for environments without gateway credentials).
- **Claim-to-code map**: `REPRODUCING.md` also carries a table mapping each paper claim to the implementing module, the test that exercises it, and the raw-results file that backs its numbers, flagging the specified-only components explicitly.

The artifact contains the AgentISA implementation (`agentflow.core.isa`), the microbenchmark and real-LLM benchmark suites with their raw-results files, the system-level demonstration and four-task characterization harnesses with their tests, the real-gateway compiler/hot-path harness with its offline-mock and real-gateway results, and the 43 multi-agent functional tests; the full per-file inventory is in Appendix F.5 of the supplementary material.